MCQ Exam Platform Documentation

Overview

This MCQ exam platform is designed to evaluate technical and logical aptitude through four structured sections. It captures user details and presents randomized questions from a larger pool, ensuring each student receives a unique experience. The exam is held in full-screen mode to minimize distractions and cheating. Anti-cheating measures like tab switch detection and screen deactivation logging are included. Upon completion, the platform computes scores and displays section-wise results to the user. All activity is recorded in Firebase Firestore, allowing for efficient data handling, reliability, and scalability to support hundreds of students simultaneously with minimal technical complexity.

Core Features

The platform offers essential exam functionalities including user registration, question fetching, response submission, and scoring. It supports four sectionsAptitude, Theory (MySQL, React, Node, Express), Code Fill-in-the-Blanks, and Debugging. Each section has a defined question format and marking scheme. The system enforces full-screen mode during the exam and tracks any tab switching or window deactivation as violations. All data is stored and retrieved using Firebase Firestore. Post-submission, students can view their responses, correct answers, and scores section-wise. This complete workflow ensures smooth, fair, and scalable online testing without needing traditional server infrastructure.

Tech Stack

The platform uses a modern, lightweight tech stack centered around React for the frontend and Firebase for backend services. React allows for rapid development and component-based UI organization, while Tailwind CSS is used for fast, responsive styling. Firebase Firestore acts as a real-time NoSQL database, ideal for storing dynamic question and response data. Firebase Hosting provides secure and fast global content delivery. Optional Firebase Cloud Functions offer serverless processing, such as backend scoring. The entire stack is cost-effective, scalable, and fully managed, minimizing server maintenance and offering an ideal solution for exam hosting platforms that prioritize performance and security.

System Architecture

This platform follows a serverless architecture pattern. The React frontend interfaces with Firebase for all

backend operations. Firebase Hosting serves the app globally. Firestore holds collections like `questions`, `users`, and `responses`. When a user begins an exam, questions are fetched from Firestore and rendered dynamically. Upon completion, their responses and scores are written back to Firestore. Firebase Functions (optional) can be used for tasks like scoring and validation on the backend. This architecture is scalable, resilient, and minimizes operational overhead. It allows for high performance and low latency with near-instant deployment using Firebases CI/CD and versioning tools.

Firestore Schema Design

Firestore organizes data in collections and documents. For this system, three primary collections are used:

1. `users`: Stores student information (name, email, phone, timestamps).

2. `questions`: Stores sets of 100 questions per section, either in one document or batched documents.

3. `responses`: Stores each students submitted answers, section-wise scores, and final submission timestamp.

Firestores indexing and real-time updates make it ideal for fast, scalable data operations. The document-based schema ensures flexibility for adding more metadata, tracking cheating flags, or extending question types. Best practices include limiting nested objects, avoiding deeply recursive structures, and using client-side filtering when feasible.

Frontend Structure

The frontend codebase is modular and organized for maintainability and scalability. Main folders include:

- `components/`: Contains reusable UI elements like question cards, timers, buttons.
- `pages/`: Includes Home, Exam, and Result pages.
- `utils/`: Helper functions for scoring, tab detection, shuffling, etc.
- `firebase/`: Firebase config and service setup.
- `App.js` and `index.js`: Entry point and router configuration.

Tailwind CSS is used for styling, with consistent utility classes. State management uses `useState`, `useEffect`, and `Context API` or Redux if needed. The routing system allows section-wise navigation, and all critical operations are encapsulated in dedicated components for reusability.

User Flow

MCQ Exam Platform Documentation

The user journey is linear and intuitive. Students start at the Home page, where they enter their name, email, and phone. Once submitted, the platform initiates the test. The exam consists of four consecutive sections: Aptitude, Theory, Code Fill-in-the-Blanks, and Debugging. Each section presents a limited number of randomized questions. Users must stay in full-screen mode; any tab switch triggers warnings or logs. Upon completion, students click Submit, and their responses are validated, scored, and stored in Firestore. The Result page then displays section-wise marks, correct answers, and optionally highlights mistakes, providing instant feedback and transparency.

Security & Anti-Cheating

To maintain exam integrity, the platform includes several anti-cheating features:

- Full-screen enforcement using browser APIs
- Detection of tab switches and window deactivation events
- Logging of suspicious activities (with timestamps)
- Randomized question order per user
- Disabling copy-paste and right-click during the exam
- Timer enforcement with auto-submit on expiry

These measures discourage and detect cheating without requiring external software or hardware. Advanced features like screen recording or webcam detection can be added later, but the current setup provides effective passive monitoring. Violations can be stored in Firestore and optionally used to flag or disqualify submissions if needed.

Suggestions for Improvement

To further enhance system reliability and security:

1. Use Firebase Functions for server-side scoring to prevent client tampering.
2. Encrypt answer submissions before storing them.
3. Implement device fingerprinting or IP tracking for anomaly detection.
4. Block paste and inspect element to prevent answer leaks.
5. Track question visit order and time per section.
6. Implement admin dashboard for question uploading, monitoring, and analytics.
7. Consider webcam snapshots or AI-based proctoring tools.

8. Store question banks in versioned documents to prevent leaks.

These measures provide a balance of user privacy, test security, and administrative control for higher-stakes deployments.

Deployment Process

Deployment is handled via Firebase CLI. First, initialize your Firebase project with `firebase init`, enabling Hosting and Firestore. Build the React app using `npm run build`, which outputs production files to the `/build` folder. Firebase Hosting is configured to serve this folder. Finally, deploy using `firebase deploy`. For custom domains, verify and map them via Firebase Console. CI/CD integration is also possible with GitHub Actions. Hosting includes automatic SSL, version rollback, and cache control. Firestore indexes and Firebase rules should be finalized before launch. Optional: deploy Firebase Functions for backend logic like scoring, cheating logs, or admin tools.

Optional Firebase Function

A Firebase Function can be created to handle scoring securely on the backend. This reduces the risk of tampering on the client side. You can create a function that accepts submitted answers, validates them against the correct set stored in Firestore, calculates scores, and writes the result to the `responses` collection. This approach centralizes the logic, simplifies testing, and adds a security layer. Functions also support HTTP endpoints, which can be restricted using Firebase Auth or tokens. Billing is still free under Spark Plan as long as you stay within 125,000 calls/month, which is far beyond your estimated usage.